MIDTERM EVALUATION

StudySync

The AI-Powered Study Assistant for CS Students

Flutter FastAPI PostgreSQL

The Problem

- → **Fragmented Information:** Notes are scattered across notebooks, PDFs, and different devices.
- → **Static Study Materials:** Searching for specific concepts in handwritten notes is impossible.
- → **Generic AI Tutors:** Existing AI tools don't understand the context of what a specific student has actually learned.
- → **Isolated Learning:** Students struggle with difficult CS concepts (OOP, DSA) alone.

Enter StudySync

A centralized ecosystem transforming passive notes into an interactive knowledge base.

🧠 Smart Capture

Convert physical notes to digital via advanced OCR.

🔍 Semantic Search

Search notes by meaning, not just exact keywords.

🤖 AI Tutor (RAG)

Get answers grounded in your own notes and CS books.

🌐 Sync & Community

Backup to Google Drive and collaborate on forums.

Core Feature 1: OCR & Digitization

Powered by Microsoft Kosmos-2.5 Vision Model

- → Students capture handwritten or printed notes via camera.
- → Image sent to Python OCR microservice (FastAPI).
- → Extracts structured text blocks with bounding boxes.
- → Saved locally via ObjectBox NoSQL database.

Core Feature 2: Semantic Search

Finding context, not just keywords.

- → Extracted OCR text is split into chunks using LangChain.
- → Chunks converted to 384-dimensional float vector embeddings.
- → Stored in ObjectBox (HNSW index) for lightning-fast on-device search.
- → "How does polymorphism work?" finds related OOP notes instantly.

Core Feature 3: AI Tutor (RAG)

Retrieval-Augmented Generation tailored for CS subjects (Programming, OOP, DB, DSA).

1. Retrieve

Fetches top-5 relevant chunks from local notes + Supabase pgvector (course books).

2. Augment

Injects context into the prompt with a "CS Professor" persona.

3. Generate

Ollama local LLM (Phi model) generates markdown-formatted responses.

Core Feature 4: Sync & Community

☁️ Cloud Sync

- → Google Drive API Integration.
- → Notes serialized to binary archives.
- → Bidirectional sync with conflict resolution.
- → Accessible across multi-platform devices.

🤝 Community Forums

- → Peer-to-peer learning environment.
- → Discuss complex CS topics.
- → Ask questions and share knowledge.

Architecture & Tech Stack

Frontend (App)

- Flutter (Dart) UI
- ObjectBox (Local Vector DB)
- Google Drive REST API

Backend (AI Services)

- Python FastAPI Microservices
- Ollama (Local LLM Container)
- PyTorch & Transformers

Cloud Backend

- Supabase (Auth)
- PostgreSQL + pgvector
- Global Vector Search

Live Demo

Project Walkthrough

Note Capture → OCR → Semantic Search → AI Tutor

[Switch to Screen Share / Video]

Code Deep Dive: Data Flow

How the AI Tutor synthesizes knowledge:

```
User Request → ask_service.py (Port 8002)

1. Embed() → Converts Question to Vector
2. Query() → Local notes (ObjectBox HNSW) + Supabase
3. Assemble() → Top-5 context chunks combined
4. Generate() → Sent to Ollama LLM container
5. Stream() → Markdown response to Flutter UI
```

Work Completed So Far

- → ✅ Complete Flutter UI Architecture & Theming.
- → ✅ Python Microservices (OCR, Embedding, RAG pipeline) orchestrated via Docker.
- → ✅ ObjectBox integration for local HNSW vector search.
- → ✅ Supabase authentication and pgvector setup.
- → ✅ Bi-directional Google Drive sync logic.

Plan of Work: Next Steps

📎 Attachments in Forums

Enhancing the community aspect by allowing students to upload and share images/snippets directly in forum discussions.

📦 App Packaging

Compiling the Flutter application into a production-ready standalone APK for Android deployment and testing.

Plan of Work: Deployment

🐳 Docker Deployment

Deploying the 4 core Python/AI containers (OCR, Embed, Ask, Ollama) to a remote cloud server for global access.

✨ Quality of Life

UI/UX polish, bug fixes, query speed optimization, and improving the AI tutor's system prompt accuracy.

Q&A

Thank you!